TP4: Quantum Error Correction

Foreword

Welcome to TP4. So far, you have lived in the friendly fiction that qubits behave themselves. They do not. Real qubits decohere within microseconds, get bumped by stray electromagnetic fields, and generally treat your carefully designed algorithms as suggestions. The whole field of quantum error correction (QEC) exists because we eventually realized that fault-tolerant quantum computing was going to be a multi-decade engineering battle, not a weekend project.

The good news: there is a beautiful theory behind QEC that takes its cue from *classical* error correction, where you protect a bit by sending it three times and majority-voting at the end. The bad news: you cannot directly copy a qubit (thank you, no-cloning theorem), measurement collapses superpositions, and errors are continuous rather than discrete. The genius of QEC is that despite all three of these obstacles, the classical idea still works. You will see why in this TP.

We build up in three layers. Section 1 protects against *bit flips* (the easy half). Section 2 protects against *phase flips* (the same algorithm in a different basis). Section 3 combines them into Shor's nine-qubit code, which protects against *any* single-qubit error. Section 4 turns you loose to benchmark or invent.

A few survival tips:

- **No-cloning is not the obstacle you think it is.** You cannot duplicate $\alpha|0\rangle + \beta|1\rangle$ into two separate qubits each holding the same state. But you *can* entangle three qubits so the combined state is $\alpha|000\rangle + \beta|111\rangle$. That is what encoding does, and it is enough.
- **Syndrome measurement is the heart of QEC.** You will use ancilla qubits (or extract bits from the data qubits themselves) to learn *which* kind of error happened, without learning *anything* about the actual logical state. This selective ignorance is what keeps the superposition alive.
- **Reuse, reuse, reuse.** The phase flip code is the bit flip code in a different basis. The Shor code is the phase flip code with each qubit replaced by a bit flip code. The whole TP is a single idea applied recursively. Resist rewriting from scratch at each step.
- **Continuous errors? Don't panic.** The math will show that you only need to handle three discrete error types (X, Y, Z), and arbitrary single-qubit errors automatically reduce to those when you measure the syndrome. This is the *discretization of errors* theorem, and it is one of the most surprising results in all of physics.

Each exercise comes with a **Hint** box. Appendix A explains *why* QEC works at all, including the no-cloning workaround and the discretization-of-errors miracle.

1 Bit Flip Error Correction

These exercises are found in the `SimpleQEC.qs` file.

In quantum computing, qubits are extremely sensitive to environmental disturbances. Two main families of errors can occur:

- **Bit flip:** $|0\rangle$ becomes $|1\rangle$ and vice versa, equivalent to applying an X gate.
- **Phase flip:** the relative phase changes sign, for example $|+\rangle$ becomes $|-\rangle$, equivalent to applying a Z gate.

To correct these, we use *repetition codes* that encode one *logical* qubit across multiple *physical* qubits. The 3-qubit repetition code for bit flips encodes a logical state $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ as:

$$|0\rangle_L \longrightarrow |000\rangle \tag{1}$$
$$|1\rangle_L \longrightarrow |111\rangle \tag{2}$$

so the encoded superposition is $\alpha\,|000\rangle + \beta\,|111\rangle$. Note that this is *not* three copies of $\alpha\,|0\rangle + \beta\,|1\rangle$ (that would violate no-cloning); it is an entangled three-qubit state.

1.1 Bit Flip Encoding

The encoding transforms a single logical qubit into three physical qubits that are entangled together: if the logical qubit was $|0\rangle$, all three become $|000\rangle$; if $|1\rangle$, all three become $|111\rangle$; superpositions extend by linearity.

Implement `EncodeBitFlip(logical: Qubit, aux: Qubit[]) : Unit is Adj` in `SimpleQEC.qs`. The `aux` array contains two qubits initially in $|0\rangle$.

Hint. You need to create correlations between the three qubits without measuring them, and without cloning (which is forbidden). Ask yourself: among the two-qubit gates you know, which one shares the value of a control qubit with a target qubit while leaving superpositions undisturbed on the control? Once you have it, the encoder is a very short sequence.

1.2 Bit Flip Decoding and Correction

Decoding must detect whether an error occurred and correct it *without destroying the quantum information*. The trick is called **syndrome extraction**: we use the auxiliary qubits to learn whether an error has happened, but in a way that does not reveal anything about α or β.

Implement `DecodeBitFlip(logical: Qubit, aux: Qubit[]) : Unit` that detects and corrects a single bit flip error on any of the three qubits.

The general recipe:

1. **Syndrome extraction.** Use CNOT gates to push information about whether the qubits agree onto the auxiliary qubits.
2. **Error correction.** The pattern of auxiliary values (the syndrome) tells you which qubit, if any, was flipped. Apply the appropriate correction.

Hint. Think about what should happen in the no-error case: the three qubits agree. Can you design a sequence that pushes any *disagreement* between the logical qubit and the auxiliaries onto the auxiliaries themselves, leaving the logical qubit alone?

Once the auxiliaries carry that information, a single multi-controlled gate can apply the correction. Drawing the truth table on paper for the four cases (no error, error on qubit 0, on qubit 1, on qubit 2) is the fastest way to see which gate.

2 Phase Flip Error Correction

These exercises are also in `SimpleQEC.qs`.

Phase flips (Z errors) are more subtle: they do not change $|0\rangle$ or $|1\rangle$, but they flip the sign of the relative phase in superposition. For example:

$$Z(\alpha\,|0\rangle + \beta\,|1\rangle) = \alpha\,|0\rangle - \beta\,|1\rangle$$

The key idea. In the Hadamard basis $\{|+\rangle, |-\rangle\}$, the roles of X and Z swap: $HZH = X$, so a phase flip in the computational basis is a bit flip in the Hadamard basis. We can therefore protect against Z errors by:

1. Switching to the Hadamard basis with a layer of H gates.
2. Running our existing bit flip code there.
3. Switching back with another layer of H gates.

2.1 Phase Flip Encoding

Implement `EncodePhaseFlip(logical: Qubit, aux: Qubit[]) : Unit is Adj` that protects against phase flips on any of the three qubits.

The encoded states are:

$$|0\rangle_L \longrightarrow |+\rangle\,|+\rangle\,|+\rangle\,, \qquad |1\rangle_L \longrightarrow |-\rangle\,|-\rangle\,|-\rangle$$

A Z error on any one qubit flips that $|+\rangle$ to $|-\rangle$ (or vice versa), which the bit flip code can then correct.

Hint. Re-read the intro to Section 2: phase flips become bit flips after a change of basis. So you already have all the pieces. The encoder needs to do two things, in some order: switch basis, and run bit flip encoding.

The basis change is a single-gate layer applied to all three qubits.

2.2 Phase Flip Decoding and Correction

Implement `DecodePhaseFlip(logical: Qubit, aux: Qubit[]) : Unit` that corrects phase flip errors.

Hint. A decoder undoes what an encoder did. Whatever sequence you wrote in `EncodePhaseFlip`, the decoder runs the corresponding inverse steps in the opposite order.

One piece of good news: the basis-change layer is self-inverse, so it looks identical on the way in and on the way out. The other piece is not, so make sure you use the right operation there.

3 Shor's 9-Qubit Code

These exercises are found in the `ShorCode.qs` file.

3.1 The Need for Universal Protection

The bit flip code corrects X errors and ignores Z ones. The phase flip code does the opposite. But real errors are arbitrary single-qubit unitaries, which expand as:

$$E = a \cdot I + b \cdot X + c \cdot Y + d \cdot Z$$

with $Y = iXZ$ and $|a|^2 + |b|^2 + |c|^2 + |d|^2 = 1$.

Shor's insight. You can protect against *arbitrary* single-qubit errors by stacking the two codes. The trick is hierarchical:

1. First, protect against phase flips with a 3-qubit phase code.
2. Then protect each of those qubits against bit flips with a 3-qubit bit flip code.
3. Total: 9 physical qubits protecting 1 logical qubit against any single-qubit error.

We arrange the 9 qubits in a 3×3 grid:

$$\begin{matrix} q_0 & q_1 & q_2 \\ q_3 & q_4 & q_5 \\ q_6 & q_7 & q_8 \end{matrix}$$

where each row is a bit flip code and the three rows together form a phase flip code (with q_0, q_3, q_6 acting as the "logical representatives").

Why does this work against Y errors? Because $Y = iXZ$, so a Y error is just an X error and a Z error happening on the same qubit. The bit flip layer catches the X, the phase flip layer catches the Z. Appendix A explains why the same machinery also handles continuous errors like $E = aI + bX + cY + dZ$ with arbitrary coefficients.

3.2 Shor Code Encoding

Implement `EncodeShorCode(logical: Qubit, aux: Qubit[]) : Unit is Adj` that takes one logical qubit and 8 auxiliary qubits, and produces the full 9-qubit encoding.

The intended structure:

- Phase-flip-encode the logical qubit using q_3 and q_6 as the auxiliaries.
- Then bit-flip-encode each of q_0, q_3, q_6 within its own row, using the remaining two qubits of that row.

Hint. You already have everything you need: the phase flip encoder and the bit flip encoder. The whole Shor encoder is just composing them in the right order, with the right qubit groupings.

The hierarchy described above tells you the order: which level encodes *across* rows, and which level encodes *within* each row? Think about which step produces "three logical qubits" that the other step can then operate on.

Read the comment above the operation signature carefully to know which indices of `aux` correspond to which positions in the 3×3 grid; it varies depending on the skeleton.

3.3 Shor Code Decoding

Implement `DecodeShorCode(logical: Qubit, aux: Qubit[]) : Unit` that detects and corrects any single-qubit error.

The structure mirrors the encoding, in reverse order:

- First, run `DecodeBitFlip` on each row to fix any bit flip error within that row.
- Then run `DecodePhaseFlip` on the three row representatives to fix any phase flip error between rows.

Hint. The decoder mirrors the encoder, with each step replaced by its decoding counterpart and the order reversed. Ask yourself: at the point where each layer expects to see "valid encoded data", which other layer must have been undone first?

Like the encoder, this should be a short composition of operations you have already written. Resist the urge to write new logic from scratch.

4 Benchmarking or DIY

In this section, little to no guidance is given, as always. You have two paths (and you are welcome to take both).

Option A: Benchmark. Implement random error injection in Python and measure the failure rate of each of the three codes as a function of the per-qubit error rate p. For low p, all three codes should fail with probability roughly proportional to p^2 (a single error is always corrected; only two errors at once break the code). For high p, the unprotected qubit eventually becomes *better* than the encoded one, because more physical qubits means more chances for at least two of them to be hit at once. Find the crossover point.

Option B: DIY code. Pick a code you find interesting (5-qubit perfect code, Steane code, surface code patch, etc.) and implement it. Test it the same way you tested the codes above. This is more work, but considerably more interesting if you have time.

Hint. For random error injection in Python, sample a random number per qubit and apply one of the three Pauli gates if it falls below your error threshold. A standard choice is the depolarizing channel, which spreads probability p evenly across X, Y, Z.

For the plot, fix a logical state to encode, sweep p over several decades on a log scale, and run enough shots per value to keep the statistical noise below the signal. The shape of the curve at low p should tell you something about how many errors the code can tolerate; the crossover with the unencoded baseline tells you when encoding stops helping.

Bonus: The Surface Code

This section is an open exploration meant for students who want to go further on their own. Unlike the rest of the TP, the material here is ***not covered in class****: in particular, the surface code relies on the* ***stabilizer formalism****, which is a more formal way of describing quantum codes via commuting Pauli operators. You will see enough of it below to play, but if you want a real foundation, plan some time to read about it independently. No specific function signatures are required, and no automated grader will check your output. The goal is to understand and play with the code that real fault-tolerant quantum computers (Google, IBM, etc.) actually build their machines around.*

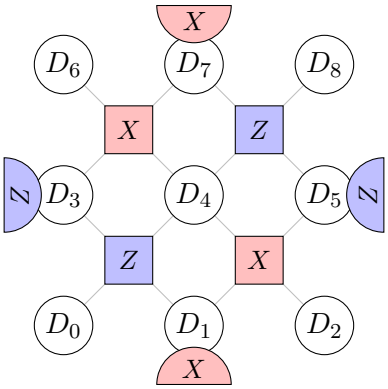
Shor's 9-qubit code corrects any single-qubit error, which is already remarkable. But it has two practical weaknesses: it does not scale (correcting two simultaneous errors would need a much bigger, ad-hoc construction), and the stabilizer measurements involve qubits that are not geometrically close (the phase-flip layer entangles qubits in different rows). On real hardware, where each qubit can only interact with its immediate neighbours, this is a deal-breaker.

The surface code solves both problems at once. It is **local** (every stabilizer involves only neighbouring qubits on a 2D grid), **scalable** (use a bigger grid to correct more errors), and has an experimentally accessible **threshold** around 1% per-gate error rate, well within reach of modern hardware. It is the workhorse of contemporary QEC research.

There is no free lunch. While Shor's code uses 9 physical qubits per logical qubit, the surface code in practice uses on the order of *a thousand or more* physical qubits per logical qubit, because the distance has to grow as you target useful algorithms. This is the painful overhead at the heart of why building a fault-tolerant quantum computer is a multi-decade engineering project.

Layout

We use the **rotated distance-3 surface code**, which is the smallest surface code that corrects one arbitrary error, and is also the variant most commonly implemented on real hardware. It uses 9 data qubits arranged in a 3×3 grid, plus 8 ancilla qubits placed in the gaps (4 of type X, 4 of type Z), for 17 qubits total.



The D_i (white circles) are the 9 data qubits, indexed by row and column. The 4 interior stabilizers (squares) each act on the 4 neighbouring data qubits. The 4 boundary stabilizers (semicircles) each act on only 2 data qubits, with the choice of which boundary carries X versus Z stabilizers being dictated by the requirement that logical operators commute with all stabilizers.

Stabilizers

Each ancilla measures one of two types of operators on its four neighbouring data qubits:

- Z-stabilizers: $Z \otimes Z \otimes Z \otimes Z$ on the four data qubits around the ancilla. These detect X errors (bit flips).
- X-stabilizers: $X \otimes X \otimes X \otimes X$ on the four data qubits around the ancilla. These detect Z errors (phase flips).

A Y error is automatically caught by both types of stabilizers, since $Y = iXZ$.

The crucial property: *all stabilizers commute with each other*, which is what makes simultaneous measurement well-defined. This is non-trivial to verify, but it boils down to the fact that any two neighbouring four-qubit Paulis share exactly two data qubits, and Paulis on the same qubit either commute (both X or both Z) or anti-commute twice (so the four-qubit operators end up commuting overall).

Logical operators

The logical $\overline{Z}$ is a string of Z operators running from the top edge to the bottom edge of the patch. The logical $\overline{X}$ is a string of X operators running from the left edge to the right edge. The shortest such strings have length d where d is the **distance** of the code (3 in our example), and the code can correct up to $\lfloor (d-1)/2 \rfloor = 1$ arbitrary error per round.

To correct more errors per round, just use a bigger patch (distance 5, 7, etc.). The number of physical qubits grows as d^2, and the failure probability falls as roughly $(p/p_{th})^{(d+1)/2}$ where $p_{th} \approx 1\%$ is the threshold. For $d = 3$ this is $(p/p_{th})^2$; for $d = 5$ it is $(p/p_{th})^3$; and so on. Doubling d does not just incrementally improve the error rate, it improves it polynomially harder.

Hint. For a Z-stabilizer measurement: prepare the ancilla in $|0\rangle$, apply a CNOT from each neighbouring data qubit (control) to the ancilla (target), then measure the ancilla. The result is the parity of the four data qubits in the Z basis: 0 if no X error happened among them, 1 if exactly one did.

For an X-stabilizer: prepare the ancilla in $|0\rangle$, apply H to it, apply CNOTs from the *ancilla* (control) to each data qubit (target), apply H again, then measure.

Caveat for the curious: the order in which you do the four CNOTs around a single ancilla matters in real fault-tolerant settings, because a bad order can let an error on the ancilla propagate to multiple data qubits and cut the effective distance in half (so-called "hook errors"). For one idealized round with no measurement errors, any order works; if you go further and chain multiple rounds, look up the standard "Z-shape" CNOT schedule.

A good reference for going deeper is Fowler, Mariantoni, Martinis & Cleland, "Surface codes: Towards practical large-scale quantum computation" (2012). The Stim and PyMatching tutorials are excellent for hands-on experimentation once you have task (1) working. Try task (1) before reading external material; it is genuinely within reach with what you already know.

There is no expected output for this section. You succeed by understanding why the surface code is structured the way it is, and by producing ***any*** *working artifact (a syndrome plot, a decoder, a benchmark curve) that shows you exercised the code. Document what you tried and what you learned; the process is the deliverable.*

A How QEC actually works (the friendly version)

Quantum error correction has three things going against it that classical error correction does not: you cannot copy qubits, you cannot measure them without destroying superpositions, and errors are continuous. This appendix walks through each obstacle and the trick that bypasses it. The math stays elementary; if you can multiply 2×2 matrices, you can follow.

Obstacle 1: no-cloning, and how entanglement replaces copying

The no-cloning theorem says there is no unitary U such that $U(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |\psi\rangle$ for all $|\psi\rangle$. So we cannot literally make three copies of $\alpha |0\rangle + \beta |1\rangle$.

But we never wanted three copies. What we wanted is a state where measuring any one of the three qubits in some clever way tells us about the original $|\psi\rangle$. The state $\alpha|000\rangle + \beta|111\rangle$ does this perfectly: it is *not* three copies (note that $(\alpha|0\rangle + \beta|1\rangle)^{\otimes 3}$ has eight terms, not two), but it is entangled in such a way that "the bit value" (whether it is the 000 branch or the 111 branch) is shared across all three qubits.

The CNOTs in your encoder do not copy quantum information. They *spread* it across qubits via entanglement. From the outside, the logical bit looks copied; from the inside, no qubit alone has any information about α or β.

Obstacle 2: measurement destroys, but syndrome measurement does not

A direct measurement of a data qubit would collapse the superposition and destroy your logical state. So how do we detect errors?

The trick is to design the measurement so that it only learns about *relations between qubits*, never about the qubits themselves. Concretely, in the 3-qubit bit flip code, the two natural questions are:

- Does q_0 agree with q_1? (Measure the parity Z_0Z_1.)
- Does q_1 agree with q_2? (Measure the parity Z_1Z_2.)

The answers are two classical bits, the **syndrome**. The key observation is that the encoded state is an eigenvector of both parity operators:

$$Z_0Z_1|000\rangle = +|000\rangle, \quad Z_0Z_1|111\rangle = +|111\rangle$$

and similarly for Z_1Z_2. So $Z_0Z_1(\alpha|000\rangle + \beta|111\rangle) = +1 \cdot (\alpha|000\rangle + \beta|111\rangle)$: the measurement returns $+1$ with certainty, and the state is undisturbed. The superposition between $|000\rangle$ and $|111\rangle$ survives, because both branches share the same eigenvalue.

These parity operators are examples of **stabilizer operators**. The formal framework that generalizes this idea (any code whose codespace is the simultaneous $+1$ eigenspace of a set of commuting Pauli operators) is called the **stabilizer formalism**. We will not develop it here, but the Shor code, the surface code, and almost every code used in practice fit in this framework, so if you ever go further into QEC, this is the name to look up.

If a single X error occurred, the parities change in a predictable way:

- Error on q_0: $Z_0Z_1 = -1$, $Z_1Z_2 = +1$. Syndrome $= (1, 0)$.
- Error on q_1: $Z_0Z_1 = -1$, $Z_1Z_2 = -1$. Syndrome $= (1, 1)$.
- Error on q_2: $Z_0Z_1 = +1$, $Z_1Z_2 = -1$. Syndrome $= (0, 1)$.
- No error: $Z_0Z_1 = +1$, $Z_1Z_2 = +1$. Syndrome $= (0, 0)$.

The syndrome *points at the error* without revealing the logical state. Apply X to the indicated qubit, and you are back to the encoded state.

In the code you write for this TP, the CNOTs in `DecodeBitFlip` extract this syndrome onto the auxiliary qubits, and the final Toffoli (or the conditioned X) applies the correction. Same idea, slightly different wiring.

Obstacle 3: discretization of errors, the central miracle

Classical errors are discrete: a bit is either flipped or not. Quantum errors are continuous: a qubit can suffer any unitary U close to identity, with infinitely many parameters. How can a finite code possibly correct an infinite family of errors?

The answer is one of the deepest results in QEC: **syndrome measurement digitizes continuous errors**.

Suppose a qubit suffers a small unitary error $E = aI + bX + cY + dZ$ (any single-qubit unitary can be written this way). The encoded state $|\psi_L\rangle$ becomes a superposition:

$$E_k |\psi_L\rangle = a |\psi_L\rangle + bX_k |\psi_L\rangle + cY_k |\psi_L\rangle + dZ_k |\psi_L\rangle$$

where X_k, Y_k, Z_k act on the k-th qubit. Each term in this sum has a different syndrome: identity gives the trivial syndrome, X_k gives a bit flip syndrome at position k, Z_k gives a phase syndrome, $Y_k = iX_kZ_k$ gives both.

When we measure the syndrome, the state collapses onto one of these four branches. We do not know which one beforehand: it is a quantum measurement with probabilities $|a|^2$, $|b|^2$, $|c|^2$, $|d|^2$. But *after* the measurement, we know exactly which error occurred (because the syndrome tells us), and we can apply the inverse to recover $|\psi_L\rangle$.

The continuous error has been forced to declare itself as one of the discrete ones. This is true even though we never knew, and never needed to know, the coefficients a, b, c, d. The Shor code's ability to correct *any* single-qubit error is not a limitation that it can only handle I, X, Y, Z. It is the fact that handling those four is automatically enough.

One-line takeaway

> *QEC trades one fragile qubit for many entangled qubits, then uses syndrome measurement to learn "which error happened" without learning "what the logical state is". Continuous errors collapse into discrete ones automatically.*

The 3-qubit codes are the smallest examples that show all the ingredients. The Shor code is the smallest example that handles arbitrary single-qubit errors. Real fault-tolerant quantum computing uses much larger codes (surface codes with thousands of physical qubits per logical qubit), but every one of them rests on these same three tricks: entanglement instead of copying, syndromes instead of direct measurement, and discretization of errors.